

Simple Query Optimizer

Testing & Evaluation Report

DBMSTeam 404

Roll Number	Name
23CS10054	Popuri Anirudh (c)
23CS10013	Chintada Yesheeth Sree Narayana
23CS10078	Vemuri Sai Lokesh
23CS10052	Payidiparthi Sri Surya Naga Hadwik (vc)
23CS10017	Dasari Veera Venkata Abhinav Teja

Department of Computer Science and Engineering
Course: Database Management Systems Lab

Contents

1	Test Suite Execution	2
1.1	Test 1: Five-Table Join with Multiple Self-Joins and DP Scaling	2
1.2	Test 2: Four-Table Join with Self-Joins and Complex Predicates	6
1.3	Test 3: Four-Table Join with Aliases and RBO Pushdown	9
1.4	Test 4: AND Predicate Split and Dual Pushdown	12
1.5	Test 5: Semantic Error Detection (Bad Column)	14

Test Suite Execution

All tests were executed on a WSL (Ubuntu 24) environment using the compiled C++ binary `optimizer_test`. The MockCatalog was loaded from `src/catalog.json`.

The following sections detail the exact SQL queries, terminal execution logs, and visualized abstract syntax trees (AST) for our five most rigorous test cases.

Test 1: Five-Table Join with Multiple Self-Joins and DP Scaling

```
SELECT u1.name, u2.name, o1.amount, o2.amount, p.name
FROM users AS u1
JOIN users AS u2 ON u1.age = u2.age
JOIN orders AS o1 ON u1.id = o1.user_id
JOIN orders AS o2 ON u1.id = o2.user_id
JOIN products AS p ON o1.order_date = p.id
WHERE u1.age > 20
      AND u2.age < 50
      AND p.category_id = 5
      AND o1.amount > 1000
      AND o2.amount > 100
      AND u1.age > u2.age
      AND o1.amount > o2.amount;
```

Listing 1: Deep Self-Join Query (test11.sql)

```
=====
      DBMSTeam 404 Optimizer Engine
=====
Parsing SQL file: test_queries/test11.sql...

--- LOGICAL EXECUTION PLAN ---
-> LOGICAL_PROJECT (u1.name, u2.name, o1.amount, o2.amount, p.name)
  -> LOGICAL_FILTER
    -> LOGICAL_JOIN (INNER)
      -> LOGICAL_JOIN (INNER)
        -> LOGICAL_JOIN (INNER)
          -> LOGICAL_JOIN (INNER)
            -> LOGICAL_GET (users AS u1)
            -> LOGICAL_GET (users AS u2)
          -> LOGICAL_GET (orders AS o1)
        -> LOGICAL_GET (orders AS o2)
      -> LOGICAL_GET (products AS p)
[Catalog] Using MockCatalog from src/catalog.json
[MockCatalog] Loaded 5 table(s) from 'src/catalog.json'.

[SUCCESS] Semantic check passed! Ready for optimization.

--- STARTING RULE-BASED OPTIMIZATION ---
[RBO] Pushing Qualified Filter down to (u1)...
[RBO] Pushing Qualified Filter down to (u2)...
[RBO] Pushing Qualified Filter down to (p)...
[RBO] Pushing Qualified Filter down to (o1)...
[RBO] Pushing Qualified Filter down to (o2)...
[RBO] Pushing Multi-Table Filter to Lowest Common Ancestor...
[RBO] Pushing Multi-Table Filter to Lowest Common Ancestor...

--- OPTIMIZED EXECUTION PLAN ---
-> LOGICAL_PROJECT (u1.name, u2.name, o1.amount, o2.amount, p.name)
  -> LOGICAL_JOIN (INNER)
    -> LOGICAL_FILTER
      -> LOGICAL_JOIN (INNER)
        -> LOGICAL_JOIN (INNER)
          -> LOGICAL_FILTER
            -> LOGICAL_JOIN (INNER)
              -> LOGICAL_FILTER
```

```

-> LOGICAL_GET (users AS u1)
-> LOGICAL_FILTER
-> LOGICAL_GET (users AS u2)
-> LOGICAL_FILTER
-> LOGICAL_GET (orders AS o1)
-> LOGICAL_FILTER
-> LOGICAL_GET (orders AS o2)
-> LOGICAL_FILTER
-> LOGICAL_GET (products AS p)

--- STARTING COST-BASED OPTIMIZATION (DP JOIN ENUMERATION) ---
[DP] Found 5 table(s) to enumerate.
[DP] Subset 0b00000011 new best cost: 875
[DP] Subset 0b00000101 new best cost: 162950
[DP] Subset 0b00000110 new best cost: 162950
[DP] Subset 0b00001001 new best cost: 162950
[DP] Subset 0b00001010 new best cost: 162950
[DP] Subset 0b00001100 new best cost: 337500
[DP] Subset 0b00010001 new best cost: 790
[DP] Subset 0b00010001 new best cost: 785
[DP] Subset 0b00010010 new best cost: 790
[DP] Subset 0b00010010 new best cost: 785
[DP] Subset 0b00010100 new best cost: 175340
[DP] Subset 0b00010100 new best cost: 162860
[DP] Subset 0b00011000 new best cost: 175340
[DP] Subset 0b00011000 new best cost: 162860
[DP] Subset 0b00000111 new best cost: 8.76634e+07
[DP] Subset 0b00000111 new best cost: 413375
[DP] Subset 0b00000111 new best cost: 400875
[DP] Subset 0b00001011 new best cost: 8.76634e+07
[DP] Subset 0b00001011 new best cost: 413375
[DP] Subset 0b00001011 new best cost: 400875
[DP] Subset 0b00001101 new best cost: 4.37503e+10
[DP] Subset 0b00001101 new best cost: 8.7838e+07
[DP] Subset 0b00001110 new best cost: 4.37503e+10
[DP] Subset 0b00001110 new best cost: 8.7838e+07
[DP] Subset 0b00010011 new best cost: 181235
[DP] Subset 0b00010101 new best cost: 7.01633e+07
[DP] Subset 0b00010101 new best cost: 355785
[DP] Subset 0b00010110 new best cost: 7.01633e+07
[DP] Subset 0b00010110 new best cost: 355785
[DP] Subset 0b00011001 new best cost: 7.01633e+07
[DP] Subset 0b00011001 new best cost: 355785
[DP] Subset 0b00011010 new best cost: 7.01633e+07
[DP] Subset 0b00011010 new best cost: 355785
[DP] Subset 0b00011100 new best cost: 7.03379e+07
[DP] Subset 0b00001111 new best cost: 4.37506e+10
[DP] Subset 0b00001111 new best cost: 1.87826e+08
[DP] Subset 0b00010111 new best cost: 7.00004e+10
[DP] Subset 0b00010111 new best cost: 7.04137e+07
[DP] Subset 0b00011011 new best cost: 7.00004e+10
[DP] Subset 0b00011011 new best cost: 7.04137e+07
[DP] Subset 0b00011101 new best cost: 7.00005e+10
[DP] Subset 0b00011101 new best cost: 1.70326e+08
[DP] Subset 0b00011101 new best cost: 1.67826e+08
[DP] Subset 0b00011110 new best cost: 7.00005e+10
[DP] Subset 0b00011110 new best cost: 1.70326e+08
[DP] Subset 0b00011110 new best cost: 1.67826e+08
[DP] Subset 0b00011111 new best cost: 7.01005e+10
[DP] Subset 0b00011111 new best cost: 5.01805e+10
[DP] Subset 0b00011111 new best cost: 4.39505e+10

[DP] Best join plan cost: 4.39505e+10

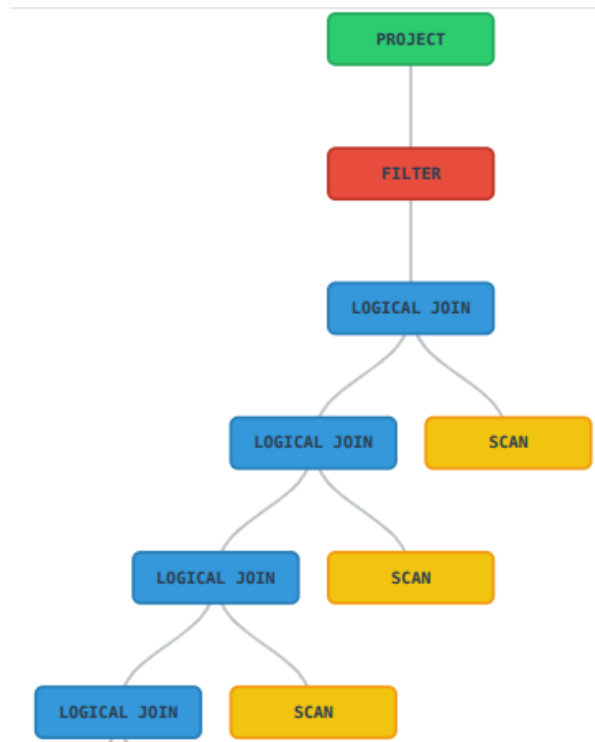
--- FINAL OPTIMIZED EXECUTION PLAN ---
-> LOGICAL_PROJECT (u1.name, u2.name, o1.amount, o2.amount, p.name)
=> [PHYSICAL_HASH_JOIN] (Memory Optimized)
-> LOGICAL_FILTER
=> [PHYSICAL_HASH_JOIN] (Memory Optimized)
-> LOGICAL_FILTER
-> LOGICAL_GET (orders AS o2)
-> LOGICAL_FILTER
-> LOGICAL_GET (orders AS o1)

```

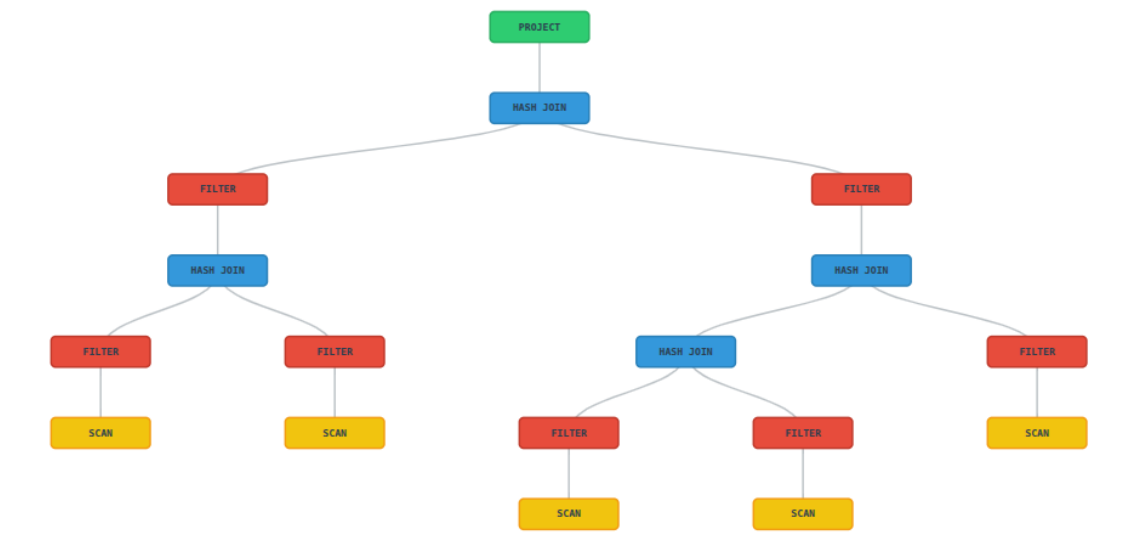
```
-> LOGICAL_FILTER
  => [PHYSICAL_HASH_JOIN] (Memory Optimized)
    => [PHYSICAL_CROSS HASH_JOIN] (Memory Optimized)
      -> LOGICAL_FILTER
        -> LOGICAL_GET (users AS u2)
      -> LOGICAL_FILTER
        -> LOGICAL_GET (products AS p)
    -> LOGICAL_FILTER
      -> LOGICAL_GET (users AS u1)

Status: Ingestion Complete!
```

Listing 2: Raw Terminal Execution Log



(a) Original Logical Tree



(b) Final Optimized Physical Tree

Figure 1: Test 1 Visual: Validates DP scaling to 5 tables across two sets of self-joins.

Test 2: Four-Table Join with Self-Joins and Complex Predicates

```
SELECT u1.name, u2.name, oo.amount, p.name
FROM users AS u1
JOIN users AS u2 ON u1.age = u2.age
JOIN orders AS oo ON u1.id = oo.user_id
JOIN products AS p ON oo.order_date = p.id
WHERE u1.age > 20
      AND u2.age > 20
      AND u1.age > u2.age
      AND oo.amount > 1000
      AND p.category_id = 5;
```

Listing 3: Self-Join Query (test10.sql)

```
=====
      DBMSTeam 404 Optimizer Engine
=====
Parsing SQL file: test_queries/test10.sql...

--- LOGICAL EXECUTION PLAN ---
-> LOGICAL_PROJECT (u1.name, u2.name, oo.amount, p.name)
    -> LOGICAL_FILTER
        -> LOGICAL_JOIN (INNER)
            -> LOGICAL_JOIN (INNER)
                -> LOGICAL_JOIN (INNER)
                    -> LOGICAL_GET (users AS u1)
                    -> LOGICAL_GET (users AS u2)
                    -> LOGICAL_GET (orders AS oo)
                -> LOGICAL_GET (products AS p)
[Catalog] Using MockCatalog from src/catalog.json
[MockCatalog] Loaded 5 table(s) from 'src/catalog.json'.

[SUCCESS] Semantic check passed! Ready for optimization.

--- STARTING RULE-BASED OPTIMIZATION ---
[RBO] Pushing Qualified Filter down to (u1)...
[RBO] Pushing Qualified Filter down to (u2)...
[RBO] Pushing Multi-Table Filter to Lowest Common Ancestor...
[RBO] Pushing Qualified Filter down to (oo)...
[RBO] Pushing Qualified Filter down to (p)...

--- OPTIMIZED EXECUTION PLAN ---
-> LOGICAL_PROJECT (u1.name, u2.name, oo.amount, p.name)
    -> LOGICAL_JOIN (INNER)
        -> LOGICAL_JOIN (INNER)
            -> LOGICAL_FILTER
                -> LOGICAL_JOIN (INNER)
                    -> LOGICAL_FILTER
                        -> LOGICAL_GET (users AS u1)
                        -> LOGICAL_FILTER
                            -> LOGICAL_GET (users AS u2)
                    -> LOGICAL_FILTER
                        -> LOGICAL_GET (orders AS oo)
            -> LOGICAL_FILTER
                -> LOGICAL_GET (products AS p)

--- STARTING COST-BASED OPTIMIZATION (DP JOIN ENUMERATION) ---
[DP] Found 4 table(s) to enumerate.
[DP] Subset 0b00000011 new best cost: 875
[DP] Subset 0b00000101 new best cost: 162950
[DP] Subset 0b00000110 new best cost: 162950
[DP] Subset 0b00001001 new best cost: 790
[DP] Subset 0b00001001 new best cost: 785
[DP] Subset 0b00001010 new best cost: 790
[DP] Subset 0b00001010 new best cost: 785
[DP] Subset 0b00001100 new best cost: 175340
[DP] Subset 0b00001100 new best cost: 162860
[DP] Subset 0b00000111 new best cost: 8.76634e+07
[DP] Subset 0b00000111 new best cost: 413375
```

```
[DP] Subset 0b00000111 new best cost: 400875
[DP] Subset 0b00001011 new best cost: 181235
[DP] Subset 0b00001101 new best cost: 7.01633e+07
[DP] Subset 0b00001101 new best cost: 355785
[DP] Subset 0b00001110 new best cost: 7.01633e+07
[DP] Subset 0b00001110 new best cost: 355785
[DP] Subset 0b00001111 new best cost: 7.00004e+10
[DP] Subset 0b00001111 new best cost: 7.04137e+07

[DP] Best join plan cost: 7.04137e+07

--- FINAL OPTIMIZED EXECUTION PLAN ---
-> LOGICAL_PROJECT (u1.name, u2.name, oo.amount, p.name)
    => [PHYSICAL_HASH_JOIN] (Memory Optimized)
        => [PHYSICAL_HASH_JOIN] (Memory Optimized)
            -> LOGICAL_FILTER
                -> LOGICAL_GET (orders AS oo)
            -> LOGICAL_FILTER
                -> LOGICAL_GET (products AS p)
        -> LOGICAL_FILTER
            => [PHYSICAL_HASH_JOIN] (Memory Optimized)
                -> LOGICAL_FILTER
                    -> LOGICAL_GET (users AS u2)
                -> LOGICAL_FILTER
                    -> LOGICAL_GET (users AS u1)

Status: Ingestion Complete!
```

Listing 4: Raw Terminal Execution Log



(a) Original Logical Tree



(b) Final Optimized Physical Tree

Figure 2: Test 2 Visual: Validates dense predicate pushdown onto 4 distinct aliases.

Test 3: Four-Table Join with Aliases and RBO Pushdown

```
SELECT *
FROM users AS u
JOIN orders AS o ON u.id = o.user_id
JOIN roles AS r ON u.role_id = r.id
JOIN products AS p ON o.id = p.id
WHERE r.role_name = 'admin';
```

Listing 5: Four-Table Join Query (test9.sql)

```
=====
DBMSTeam 404 Optimizer Engine
=====
Parsing SQL file: test_queries/test9.sql...

--- LOGICAL EXECUTION PLAN ---
-> LOGICAL_PROJECT (*)
    -> LOGICAL_FILTER
        -> LOGICAL_JOIN (INNER)
            -> LOGICAL_JOIN (INNER)
                -> LOGICAL_JOIN (INNER)
                    -> LOGICAL_GET (users AS u)
                    -> LOGICAL_GET (orders AS o)
                    -> LOGICAL_GET (roles AS r)
                -> LOGICAL_GET (products AS p)
[Catalog] Using MockCatalog from src/catalog.json
[MockCatalog] Loaded 5 table(s) from 'src/catalog.json'.

[SUCCESS] Semantic check passed! Ready for optimization.

--- STARTING RULE-BASED OPTIMIZATION ---
[RBO] Pushing Qualified Filter down to (r)...

--- OPTIMIZED EXECUTION PLAN ---
-> LOGICAL_PROJECT (*)
    -> LOGICAL_JOIN (INNER)
        -> LOGICAL_JOIN (INNER)
            -> LOGICAL_JOIN (INNER)
                -> LOGICAL_GET (users AS u)
                -> LOGICAL_GET (orders AS o)
            -> LOGICAL_FILTER
                -> LOGICAL_GET (roles AS r)
        -> LOGICAL_GET (products AS p)

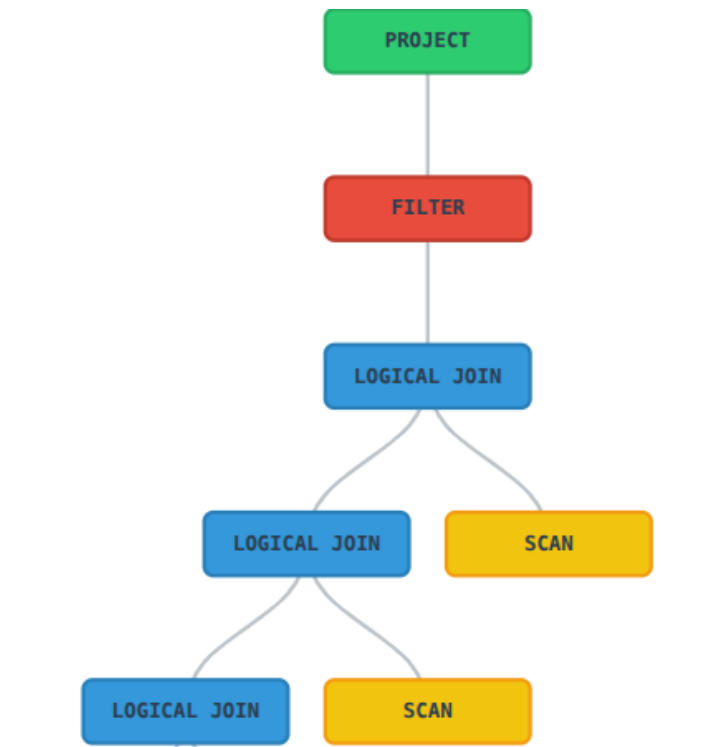
--- STARTING COST-BASED OPTIMIZATION (DP JOIN ENUMERATION) ---
[DP] Found 4 table(s) to enumerate.
[DP] Subset 0b00000011 new best cost: 162950
[DP] Subset 0b00000101 new best cost: 453.125
[DP] Subset 0b00000101 new best cost: 428.25
[DP] Subset 0b00000110 new best cost: 175003
[DP] Subset 0b00000110 new best cost: 162503
[DP] Subset 0b00001001 new best cost: 790
[DP] Subset 0b00001001 new best cost: 785
[DP] Subset 0b00001010 new best cost: 175340
[DP] Subset 0b00001010 new best cost: 162860
[DP] Subset 0b00001100 new best cost: 343.25
[DP] Subset 0b00000111 new best cost: 600453
[DP] Subset 0b00000111 new best cost: 176553
[DP] Subset 0b00000111 new best cost: 164178
[DP] Subset 0b00001011 new best cost: 7.01633e+07
[DP] Subset 0b00001011 new best cost: 355785
[DP] Subset 0b00001101 new best cost: 1693.25
[DP] Subset 0b00001110 new best cost: 176243
[DP] Subset 0b00001110 new best cost: 163843
[DP] Subset 0b00001111 new best cost: 3.50164e+08
[DP] Subset 0b00001111 new best cost: 1.07669e+06
[DP] Subset 0b00001111 new best cost: 843288
[DP] Subset 0b00001111 new best cost: 800788

[DP] Best join plan cost: 800788
```

```
--- FINAL OPTIMIZED EXECUTION PLAN ---
-> LOGICAL_PROJECT (*)
    => [PHYSICAL_HASH_JOIN] (Memory Optimized)
        => [PHYSICAL_HASH_JOIN] (Memory Optimized)
            -> LOGICAL_GET (orders AS o)
            -> LOGICAL_FILTER
                -> LOGICAL_GET (roles AS r)
        => [PHYSICAL_CROSS HASH_JOIN] (Memory Optimized)
            -> LOGICAL_GET (users AS u)
            -> LOGICAL_GET (products AS p)

Status: Ingestion Complete!
```

Listing 6: Raw Terminal Execution Log



(a) Original Logical Tree



(b) Final Optimized Physical Tree

Figure 3: Test 3 Visual: Alias resolution and RBO filter pushdown to `roles`.

Test 4: AND Predicate Split and Dual Pushdown

```
SELECT name, amount
FROM users
JOIN orders ON users.id = orders.user_id
WHERE age > 18 AND amount > 500;
```

Listing 7: AND Predicate Split Query (test8_and_filter.sql)

```
=====
DBMSTeam 404 Optimizer Engine
=====
Parsing SQL file: test_queries/test8_and_filter.sql...

--- LOGICAL EXECUTION PLAN ---
-> LOGICAL_PROJECT (name, amount)
  -> LOGICAL_FILTER
    -> LOGICAL_JOIN (INNER)
      -> LOGICAL_GET (users AS users)
      -> LOGICAL_GET (orders AS orders)
[Catalog] Using MockCatalog from src/catalog.json
[MockCatalog] Loaded 5 table(s) from 'src/catalog.json'.

[SUCCESS] Semantic check passed! Ready for optimization.

--- STARTING RULE-BASED OPTIMIZATION ---
[RBO] Catalog resolved unqualified 'age' to table (users). Pushing down...
[RBO] Catalog resolved unqualified 'amount' to table (orders). Pushing down...

--- OPTIMIZED EXECUTION PLAN ---
-> LOGICAL_PROJECT (name, amount)
  -> LOGICAL_JOIN (INNER)
    -> LOGICAL_FILTER
      -> LOGICAL_GET (users AS users)
    -> LOGICAL_FILTER
      -> LOGICAL_GET (orders AS orders)

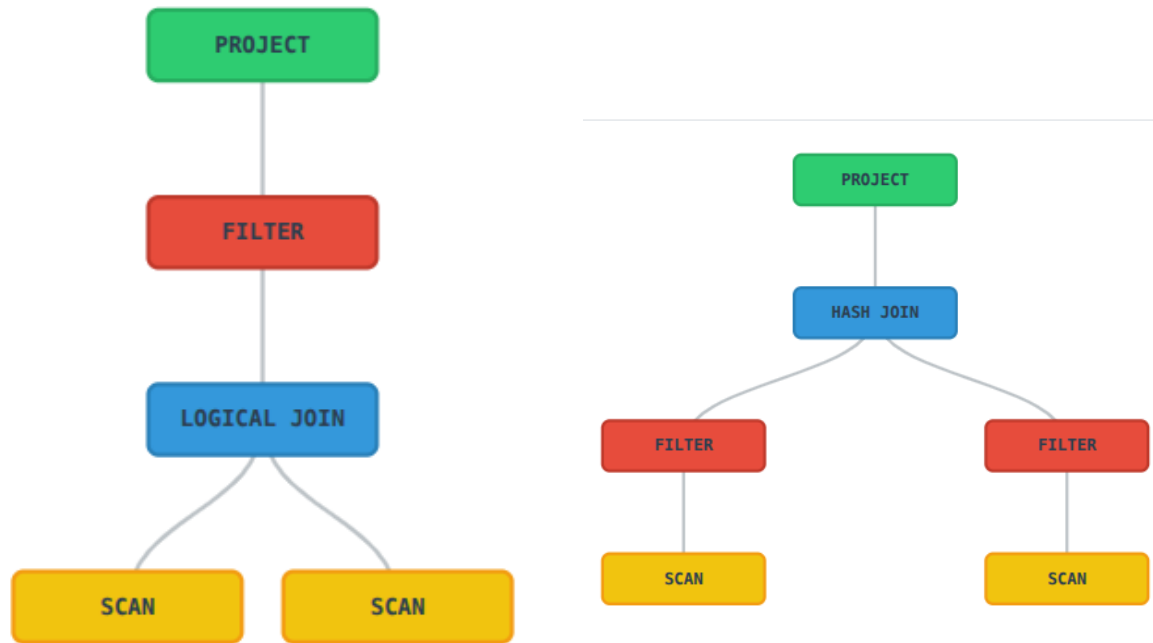
--- STARTING COST-BASED OPTIMIZATION (DP JOIN ENUMERATION) ---
[DP] Found 2 table(s) to enumerate.
[DP] Subset 0b00000011 new best cost: 162950

[DP] Best join plan cost: 162950

--- FINAL OPTIMIZED EXECUTION PLAN ---
-> LOGICAL_PROJECT (name, amount)
  => [PHYSICAL_HASH_JOIN] (Memory Optimized)
    -> LOGICAL_FILTER
      -> LOGICAL_GET (orders AS orders)
    -> LOGICAL_FILTER
      -> LOGICAL_GET (users AS users)

Status: Ingestion Complete!
```

Listing 8: Raw Terminal Execution Log



(a) Original Logical Tree

(b) Final Optimized Physical Tree

Figure 4: Test 4 Visual: Compound AND filter safely split and pushed to respective tables.

Test 5: Semantic Error Detection (Bad Column)

```
SELECT salary FROM users JOIN orders ON users.id = orders.user_id;
```

Listing 9: Bad Column Query (test7_bad_column.sql)

```
=====
DBMSTeam 404 Optimizer Engine
=====
Parsing SQL file: test_queries/test7_bad_column.sql...

--- LOGICAL EXECUTION PLAN ---
-> LOGICAL_PROJECT (salary)
    -> LOGICAL_JOIN (INNER)
        -> LOGICAL_GET (users AS users)
        -> LOGICAL_GET (orders AS orders)
[Catalog] Using MockCatalog from src/catalog.json
[MockCatalog] Loaded 5 table(s) from 'src/catalog.json'.
Semantic Error: Column not found in active tables: salary

[FAILED] Semantic check failed. Stopping execution.
```

Listing 10: Raw Terminal Execution Log

Note on Test 5: Because the semantic analyzer intentionally traps the invalid column `salary` and halts the process prior to the optimization phase, no visual tree output is generated. This confirms that our pre-optimizer safety constraints successfully prevent invalid queries from wasting CPU cycles in the DP enumerator.